USER_GUIDE

Docs Chain — User Guide

Revision: 2 **Last modified:** 2026-05-29T12:00:00Z **Status:** The Phase 1–3 engine plus the Phase-4 config loader + CLI (`doctor` / `sync` / `verify` / `graph`) are IMPLEMENTED + tested (`go test -race ./...` passes); the built `docs_chain` binary runs these subcommands today. The fsnotify `watch` daemon (§8) remains PLANNED. See status tags per section. **Authority:** Operator mandate 2026-05-29 (Docs Chain initiative) **Design provenance:** authoritative Phase-0 DESIGN / RESEARCH / PLAN live in the consuming project research tree (`docs/research/docs_chain/`); this document is the self-contained specification.

Status legend

Per §11.4.6 (no-guessing), every workflow below carries a status tag:

- **IMPLEMENTED** — the command works today.
- **PLANNED (Phase N)** — designed, not yet built (per `PLAN.md`).

The Phase 1–3 engine and the Phase-4 config loader + CLI are built and tested. The `docs_chain` binary runs `doctor` / `sync` / `verify` / `graph` today (each section below is tagged IMPLEMENTED). The only PLANNED shell workflow in this guide is the fsnotify `watch` daemon (§8); its copy-paste example shows the DESIGNED interface and is not a claim that `watch` runs today.

---

1. What you adopt Docs Chain for

If your project keeps any of these mutually-consistent, you are the audience:

- Markdown sources and their `.html` / `.pdf` exports.
- A SQLite database that is the single source of truth for documents generated from it (and editable back into it).
- Generated summaries / status docs that must never drift from their source.
- Roster / corpus fingerprints that drive a Status doc.

Docs Chain replaces the per-purpose sync scripts each of these otherwise requires (see USE_CASE_CATALOGUE.md).

---

2. Prerequisites

Status: PLANNED (Phase 4).

Requirement	Why
Go toolchain (build) OR the prebuilt `docs_chain` binary	the engine is Go
`pandoc`	`markdown→html` builtin transform
`weasyprint`	`html→pdf` / `markdown→pdf` builtin transforms
`sqlite3` (or the project's DB binary, e.g. §11.4.93 `workable-items`)	`sync` edges touching a `sqlite` node
Any `exec:` transform scripts your contexts reference	migration adapters

Docs Chain shells out to these existing commands during migration, so no rewrite of your current generators is required to start.

---

3. Install / build

Status: PLANNED (Phase 1 build / Phase 6 submodule distribution).

3.1 As a consuming-project submodule (post Phase 6)

```
# Phase 6 is OPERATOR-GATED: the remote repo + submodule wiring
        are
# created by the operator, not by an agent (§11.4.66).
git submodule add git@github.com:vasic-digital/docs_chain.git
        docs_chain
git submodule update --init --recursive
( cd docs_chain && go build -o ../bin/docs_chain ./cmd/
        docs_chain )
```

3.2 Build from a checkout

```
cd docs_chain
go build -o ./docs_chain ./cmd/docs_chain
./docs_chain doctor          # validate any contexts you have
        defined
```

---

4. Initialize `.docs_chain/`

Status: IMPLEMENTED (Phase 4 — config loader).

Docs Chain reads per-context YAML from `.docs_chain/contexts/` at your project root and writes its hash state to `.docs_chain/state.json`.

```
mkdir -p .docs_chain/contexts
```

Add to your `.gitignore` (state is regenerable per §11.4.77):

```
.docs_chain/state.json
*.docs_chain.tmp
```

The `.docs_chain/contexts/*.yaml` files ARE tracked — they are your chain definitions. `state.json` is NOT tracked — `docs_chain sync` regenerates it from the live artefacts.

---

5. Define a context

Status: IMPLEMENTED (Phase 4 — config loader).

A context is one YAML file describing the nodes and edges of one chain. Minimal example for a single Markdown doc with HTML + PDF exports (the universal markdown-export chain, §11.4.65):

```yaml
# .docs_chain/contexts/guide.yaml
context: guide
description: One markdown doc with synchronized html + pdf exports
nodes:
  guide_md:   { kind: markdown, path: docs/guides/MY_GUIDE.md }
  guide_html: { kind: html,     path: docs/guides/MY_GUIDE.html }
  guide_pdf:  { kind: pdf,      path: docs/guides/MY_GUIDE.pdf }
edges:
  - { type: derive-from, from: guide_md,   to: guide_html,
      transform: md-to-html }
  - { type: derive-from, from: guide_html, to: guide_pdf,
      transform: html-to-pdf }
transforms:
  md-to-html: { builtin: pandoc-html }
  html-to-pdf: { builtin: weasyprint-pdf }
```

The full field reference is in CONFIG_SCHEMA.md. Ready-to-use recipes for the common chains (issues, fixed, status, roster, changelog, README doc-link, CONTINUATION) are in USE_CASE_CATALOGUE.md.

---

6. Register nodes + edges (rules of thumb)

Status: PLANNED (Phase 4).

- Every artefact you want kept in sync is a **node** with a unique id and a `path`.
- A one-way generation is a **`derive-from`** edge `from → to` with a `transform`.
- A two-way single-source-of-truth relation (e.g. `markdown ⍰ sqlite`) is a **`sync`** edge `a ⍰ b` with a declared `authority`.

- The `derive-from` sub-graph MUST be acyclic — Docs Chain refuses to run on a cycle (exit 4).

---

7. Run a one-shot sync

Status: IMPLEMENTED (Phase 4 — `sync`).

```
# Sync one context
docs_chain sync guide

# Sync every registered context
docs_chain sync --all
```

Exit-code contract:

Exit	Meaning	Action
0	in-sync, or changes applied	none
2	conflict (both sides of a `sync` edge dirty)	resolve manually (§9)
3	a transform failed; run rolled back, no live changes	inspect transform output
4	cycle or config error	fix the context YAML

A run is **all-or-nothing** (ARCHITECTURE §8): on any error before commit, every live artefact and the DB are left byte-identical to the pre-run state.

---

8. Run the watch daemon

Status: PLANNED (Phase 4 — fsnotify daemon).

For interactive development, the watch daemon runs `sync` automatically on a debounced settle after edits:

```
docs_chain watch                # all contexts
docs_chain watch --context guide # one context
```

Stop it with Ctrl-C. The daemon uses fsnotify (via `vasic-digital/Watcher`) — no external daemon process is required.

---

9. Interpret and resolve conflicts

Status: PLANNED (Phase 3 conflict semantics / Phase 4 surfacing).

A conflict (exit 2) means **both** sides of a `sync` edge changed since the last sync — for example, `Issues.md` was hand-edited AND `workable_items.db` was mutated by another tool. Docs Chain writes nothing and reports which nodes are both-dirty.

To resolve (per §11.4.66 — no silent merge, no guessing per §11.4.6):

1. Decide which side is authoritative for this change.
2. Re-apply your intended change to the authoritative side only.
3. Discard / re-derive the other side from the authority.
4. Re-run `docs_chain sync <context>` — now exactly one side is dirty and the run proceeds.

Docs Chain will never auto-merge two divergent edits; that decision is yours.

---

10. CI integration

Status: IMPLEMENTED (Phase 4 `verify`); Phase 7.4 operator-gated pre-build wiring PLANNED.

Use the read-only `verify` command as a gate: it reports drift and writes nothing.

```
# In a CI step or local pre-build gate:
docs_chain verify --all || {
  echo "Tracked docs are out of sync. Run: docs_chain sync --all"
       >&2
  exit 1
}
```

`verify` is the deterministic (§11.4.50) sink-side check and the successor to the legacy `--check-only` script flags. Wiring it into `pre_build_verification.sh` is **operator-gated** (Phase 7.4) because it edits a governance-adjacent gate file.

For audit, emit the resolved graph:

```
docs_chain graph issues   # DOT/JSON of the issues chain
docs_chain doctor         # validate all contexts + state
        integrity
```

---

11. Troubleshooting

Status: PLANNED (Phase 4 — diagnostics).

Symptom	Cause	Resolution
exit 4 at load	a cycle in the `derive-from` sub-graph, or malformed YAML	run `docs_chain graph <context>`; remove the cycle or fix the field
exit 2 repeatedly		stop the second writer, resolve once (§9), re-run

Symptom	Cause	Resolution
	both `sync` sides keep changing between runs	
exit 3	a transform command failed	check the transform's stderr in `qa-results/docs_chain/<run-id>/`; run rolled back, no live damage
"node hash mismatch on a file you did not edit"	a transform is non-deterministic	the transform must produce byte-stable output (§11.4.50); fix the generator
`state.json` missing	gitignored + regenerable	a fresh `docs_chain sync` rebuilds it (§11.4.77)